



**TRƯỜNG ĐẠI HỌC
CÔNG NGHỆ ĐÔNG Á**
EAST ASIA UNIVERSITY OF TECHNOLOGY

International Training & Cooperation Institute

Introduction to Blockchain Technology

Lesson 7 – Ethereum Fundamentals and Accounts



Ethereum Fundamentals and Accounts



- 1. Ethereum Origin and Design Motivation**
- 2. Bitcoin vs Ethereum: System Differences**
- 3. Ethereum Data Model**
- 4. Ethereum Accounts and Addressing**
- 5. Transactions and State Transitions**
- 6. Summary**
- 7. Discussion**

1. Ethereum Origin and Design Motivation

Ethereum was proposed in **2013** by **Vitalik Buterin**.

- ❖ Bitcoin is powerful but limited
 - ❖ Many applications cannot be built easily on Bitcoin
- ⇒ Create a blockchain that supports **general-purpose applications**

Ethereum launched in **2015**.



1. Ethereum Origin and Design Motivation

Bitcoin was **designed** mainly for:

- ❖ Secure digital payments
- ❖ Simple and predictable validation
- ❖ Limited computation for safety

Limitations of Bitcoin:

- ❖ No built-in support for applications
- ❖ No persistent application data
- ❖ Very restricted logic

These limitations are **by design**, not mistakes.

Ethereum was created to:

- ❖ Support programmable logic on blockchain
- ❖ Allow applications to run on-chain
- ❖ Maintain shared application state

1. Ethereum Origin and Design Motivation

From Transactions to State

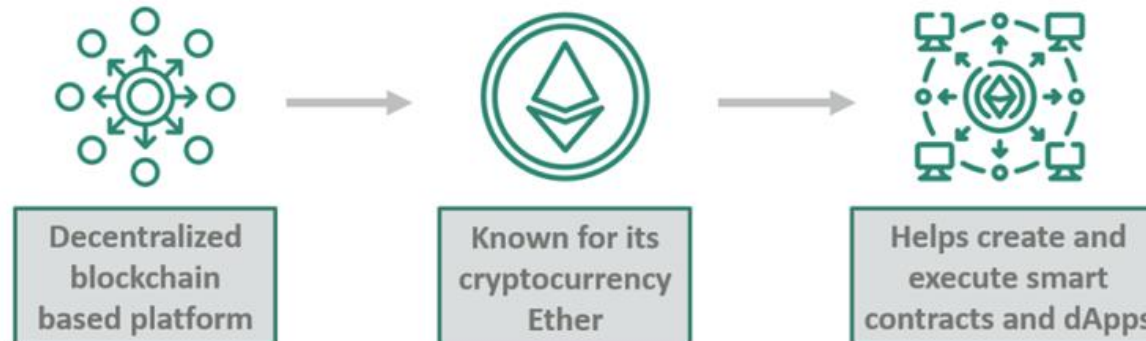
Bitcoin Perspective

- ❖ Blockchain = list of transactions
- ❖ Focus: *Where did coins come from?*
- ❖ State is reconstructed from UTXOs

Ethereum Perspective

- ❖ Blockchain = sequence of state updates
- ❖ Focus: *What is the current system state?*

This is a **fundamental design shift**.



1. Ethereum Origin and Design Motivation

Ethereum Computation:

- ❖ Executing logical instructions
- ❖ Reading and writing stored data
- ❖ Updating account balances and contract storage

Ethereum treats blockchain as a distributed system that executes programs and updates shared state.

Example

Payment system → update balances

Voting system → store votes and update results

Token system → track ownership and transfers

Q: Is computation risky on a blockchain?

1. Ethereum Origin and Design Motivation

Why Gas is needed?

- ❖ Problem: Ethereum is **Turing-complete**(supports loops), a program could run forever by accident or attack.
- ❖ Solution (Gas): Gas is a metering mechanism. Every instruction (addition, storage, etc.) has a fixed cost
- ❖ Execution Limit: If a transaction runs out of gas, the EVM stops it and cancels all changes.

1. Ethereum Origin and Design Motivation

Ethereum Account: a state record.

An account stores:

- ❖ Address (identifier)
- ❖ Balance (ETH)
- ❖ Nonce (counter)
- ❖ Optional code
- ❖ Optional storage

An account represents the current status of an entity in the system.

Example *Address: 0xA1B2..., Balance: 6 ETH, Nonce: 4*
⇒ This account owns 6 ETH. It has sent 4 transactions

Data is stored directly, not calculated.

Q: Is an account a transaction?

2. Bitcoin vs Ethereum: System Differences

Bitcoin vs Ethereum

❖ Design goals are different

	Bitcoin	Ethereum
State unit	UTXO	Account
Balance	Calculated	Stored directly
Data storage	Transaction-based	State-based
Application data	Not supported	Supported

2. Bitcoin vs Ethereum: System Differences

Validation Logic vs Application Logic

Bitcoin: Decide whether a transaction is valid

- ❖ Logic is used only to validate transactions
- ❖ No stored variables
- ❖ No repeated execution
- ❖ Logic always ends quickly

Ethereum: Define how the system behaves over time

- ❖ Logic controls how state changes
- ❖ Can store data permanently
- ❖ Can execute multiple steps
- ❖ Used to build applications

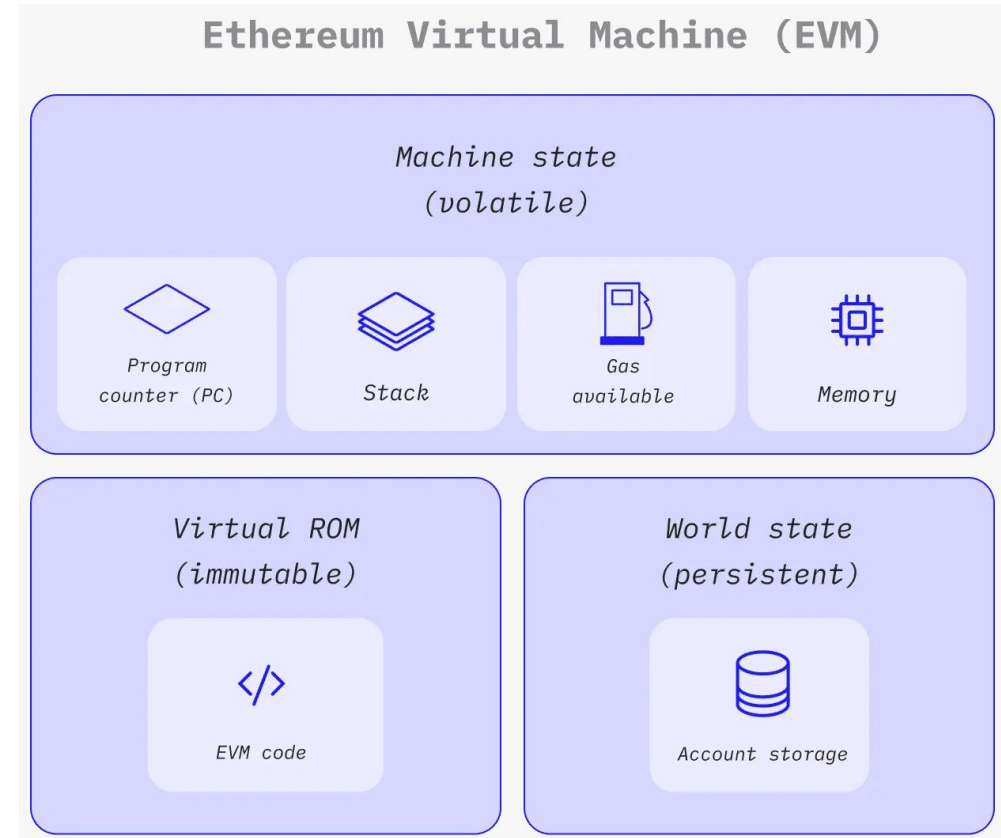
3. Ethereum Data Model

Ethereum Virtual Machine (EVM)

- ❖ A software execution environment
- ❖ Used by every Ethereum full node
- ❖ Responsible for executing smart contract code

What the EVM Does

- ❖ Reads contract bytecode
- ❖ Executes instructions step by step
- ❖ Updates account state and storage
- ❖ Produces deterministic results



3. Ethereum Data Model

Bitcoin Script

- ❖ No loops
- ❖ No persistent variables
- ❖ Used only for transaction validation

Example: *Is this signature valid?*

Ethereum Bytecode

- ❖ Executed by the Ethereum Virtual Machine (EVM)
- ❖ Supports loops, conditions, and function calls
- ❖ Can modify persistent storage

Example: *If condition X, update data Y*

3. Ethereum Data Model

Every node runs the same EVM logic.

Without EVM:

- ❖ Different machines would execute code differently
- ❖ Results would diverge
- ❖ Consensus would fail

⇒ EVM standardizes execution across the network.

Q: Why can't nodes just execute smart contracts directly on their OS?

3. Ethereum Data Model

- ❖ Ethereum is defined as a **state transition system**.
- ❖ Every node runs the EVM
- ❖ Every node executes the same bytecode

- ❖ State: all accounts and their data
- ❖ Transactions: inputs
- ❖ Execution: state transition

New State = Apply(Transaction, Current State)

Q: Why must Ethereum state transitions be deterministic?

3. Ethereum Data Model

Ethereum maintains a **global state**:

- ❖ A snapshot of all accounts
- ❖ Includes balances, nonces, contract data
- ❖ Similar to a database snapshot, updated after each transaction

Every full node:

- ❖ Stores the global state
- ❖ Independently updates it

4. Ethereum Accounts and Addressing

Hashing in Ethereum

Keccak-256 is a cryptographic hash function.

- ❖ Takes any input
- ❖ Produces fixed-length output
- ❖ Is deterministic
- ❖ Is hard to reverse

Ethereum uses Keccak-256 to:

- ❖ Derive addresses: Public Key $\rightarrow$ Keccak-256 $\rightarrow$ Last 20 Bytes $\rightarrow$ Hexadecimal (Starts with 0x)
- ❖ Ensure data integrity

4. Ethereum Accounts and Addressing

Ethereum address:

- ❖ 20 bytes (160 bits)
- ❖ Derived from a public key

Address creation steps:

- ❖ Generate key pair
- ❖ Take public key
- ❖ Apply **Keccak-256 hash**
- ❖ Take the last 20 bytes

Public key → Keccak-256 → last 20 bytes → address

4. Ethereum Accounts and Addressing

Account Types in Ethereum

Ethereum defines **two account types**:

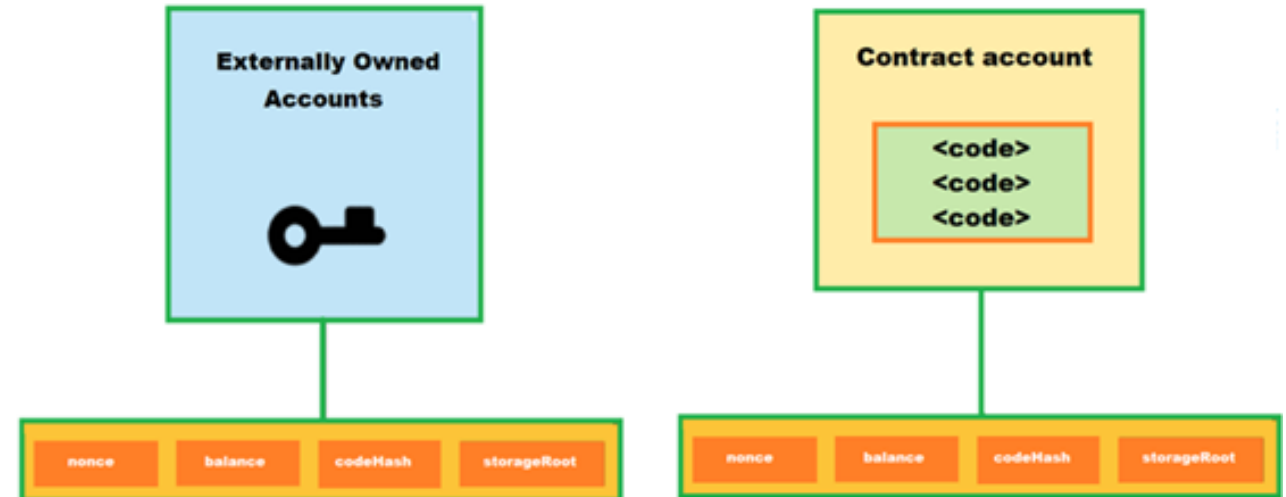
- ❖ Externally Owned Account (EOA)
- ❖ Smart Contract Account

Both:

- ❖ Have addresses
- ❖ Can hold ETH
- ❖ Exist in global state

Only EOAs can start transactions.

Externally Owned Account Vs Contract Account



4. Ethereum Accounts and Addressing

Externally Owned Account (EOA)

- ❖ Controlled by a private key
- ❖ Can initiate transactions
- ❖ Has no code or storage

Example: *MetaMask wallet*



4. Ethereum Accounts and Addressing

Smart Contract Accounts

- ❖ Contain code
- ❖ Have persistent storage
- ❖ Cannot initiate transactions
- ❖ Triggered only by:
 - EOAs
 - Other smart contracts

Contracts run **only when called**.

Smart contracts do not run automatically.

Q: Why can EOAs, but not Smart Contracts, initiate transactions?

5. Transactions and State Transitions

Ethereum Nonce (EOA)

- ❖ Counts transactions sent by an EOA
- ❖ Prevents replay attacks
- ❖ Enforces transaction order
- ❖ All user wallets are EOAs

Example: *Current nonce = 5*

Next transaction must use nonce = 5

After execution → nonce becomes 6

Q1: What happens if the nonce is incorrect?

Q2: An attacker copies a valid transaction from the blockchain: "Pay 1 ETH to Attacker, Nonce: 10." They try to broadcast it again. What will happen?

5. Transactions and State Transitions

Deterministic execution:

- ❖ Same input
- ❖ Same code
- ❖ Same blockchain state
 - Same result on all nodes

Ethereum nodes do not trust each other.

All nodes must compute the same state.

What Contracts Cannot Depend On

- ❖ Local system time: different on each machine
- ❖ Internet data: external and inconsistent
- ❖ True randomness: non-reproducible

Q: Can smart contracts call external APIs?

5. Transactions and State Transitions

Allowed alternative

- ❖ Block-provided values (e.g., *block.timestamp*)
- ❖ Stored blockchain data

Q: Why is *block.timestamp* allowed but system time is not?

5. Transactions and State Transitions

Ethereum Transactions

An Ethereum transaction:

- ❖ Comes from an EOA
- ❖ May:
 - Transfer ETH
 - Call a contract function
- ❖ Always causes a **state change**

Transaction includes:

- ❖ Sender
- ❖ Receiver
- ❖ Value
- ❖ Data
- ❖ Gas limit: Maximum gas the sender allows

5. Transactions and State Transitions

Ethereum Account Mechanics (EOA vs. Smart Contract)

Feature	Externally Owned Account (EOA)	Smart Contract Account
Controller	Human user with a Private Key.	Program Code deployed to the EVM.
Action	Active: Can initiate transactions.	Passive: Only acts when called by an EOA.
Components	Address, Balance, Nonce.	Address, Balance, Nonce, Code, Storage.
Cost	Creation is free	Creation costs Gas (deployment)

5. Transactions and State Transitions

Ethereum State Transition Flow

- ❖ Verify signature
- ❖ Check nonce
- ❖ Reserve gas for execution
- ❖ Execute code (if any)
- ❖ Update balances and storage
- ❖ Finalize new state

6. Summary

- ❖ Ethereum extends blockchain to computation
- ❖ Uses a global account-based state
- ❖ EOAs control actions via private keys
- ❖ Smart contracts store code and data
- ❖ Transactions deterministically update state
- ❖ Ethereum enables decentralized applications

7. Discussion

1. Why must all Ethereum nodes execute every transaction?
2. Why is Ethereum more suitable for applications than Bitcoin?
3. What new risks does Ethereum introduce compared to Bitcoin?
4. Why can't smart contracts access the internet directly?